2025 WINTER SEMESTER
Programming for Data Science (BCSE207P) Lab Record
Exercise Program(EP)

Date:10/3/26 **Week No.:7** **Program S. No. 33**

Title	Patient Diagnostic Analytics Using R Data Frames

Problem Statement

You are assisting a hospital's quality cell to analyze routine diagnostic test results for out-patients. For each of **10 patients**, four standardized diagnostic test scores (scaled 0–100) have been recorded. Your tasks are to:

1. **Create a vector for each column**,
2. **Combine them into a single data frame**,
3. **Compute patient-level** Sum, Average, and Rank,
4. **Compute test-wise averages** (across all patients), and
5. **Compute department-wise averages** (per test and overall).

Important: Use **only base R** and **data frames**. **No external packages** (e.g., dplyr, data.table, tibble) and **no file I/O**.

Dataset (10 Patients)

Create vectors **exactly** for the following columns and values:

PatientID	Name	Department	Diagnosis	Test1	Test2	Test3	Test4	Test5
HOS1001	Arjun Menon	Cardiology	Hypertension	82	76	88	84	88
HOS1002	Bhavana Iyer	Neurology	Migraine	71	69	73	75	73
HOS1003	Chirag Gupta	Endocrinology	Type-2 Diabetes	90	92	86	91	86
HOS1004	Devika Nair	Pulmonology	Asthma	64	70	67	72	67
HOS1005	Eshwar Rao	Cardiology	Hypertension	95	94	93	96	93
HOS1006	Farida Sheikh	Endocrinology	Thyroid Disorder	60	63	61	66	61
HOS1007	Gaurav Kumar	Neurology	Epilepsy	78	82	79	81	79
HOS1008	Harini Krishnan	Pulmonology	COPD	68	72	70	69	70

HOS 1009	Ishita S	Cardiology	Arrhythmia	87	85	90	88	90
HOS 1010	Jatin Verma	Endocrinology	Type-1 Diabetes	83	79	77	85	77

Tasks & Deliverables

Task 1 — Create Column Vectors

- Create one vector **per column** named exactly:
  PatientID, Name, Department, Diagnosis, Test1, Test2, Test3, Test4, Test3_Score.
- **Type rules:** character for IDs/names/department/diagnosis; numeric for tests.

Deliverable: Code that constructs all nine vectors.

Task 2 — Build the Data Frame

- Combine the vectors into a data frame named **patients** with stringsAsFactors = FALSE.
- Show str(patients) and head(patients).

Deliverable: The patients data frame printed to console; str() and head() outputs.

Task 3 — Patient-Level Sum, Average, Rank

1. **Sum:** Add Sum = Test1 + Test2 + Test3 + Test4.
2. **Average:** Add Average = Sum / 4 (optionally display with round() for readability).
3. **Rank:** Add Rank such that **highest Sum = rank 1**.
   - Choose and **state** your tie strategy (e.g., ties.method = "min" or "dense").
   - Example: patients$Rank <- rank(-patients$Sum, ties.method = "min").

Deliverables:

- Full data frame with Sum, Average, and Rank.
- Sorted view by Rank ascending: patients[order(patients$Rank),].
- A one-line note describing tie handling.

Constraint: Use **vectorized operations** or rowSums(); **no loops**.

Task 4 — Test-Wise Average (Overall)

Compute the **overall average** for each diagnostic test (Test1–Test4) across all patients.

Deliverable: A clearly printed summary (named vector or compact table), e.g.:
Test1: xx.xx, Test2: xx.xx, Test3: xx.xx, Test4: xx.xx.

Hint (base R):

```
test_cols <- c("Test1","Test2","Test3","Test4")
test_wise_avg <- colMeans(patients[ , test_cols], na.rm = TRUE)
```

Task 5 — Department-Wise Averages (Per Test + Overall)

For each **Department**, compute the mean of **each test** and add an **Overall_Average** (mean across the four tests). Present a tidy summary; optionally sort by Overall_Average (descending).
Deliverable: A table with columns:
Department, Test1, Test2, Test3, Test4, Overall_Average.
Hint (base R aggregate):

```
dept_test_avg <- aggregate(
  patients[ , test_cols],
  by = list(Department = patients$Department),
  FUN = function(x) mean(x, na.rm = TRUE)
)
dept_test_avg$Overall_Average <- rowMeans(dept_test_avg[ , test_cols], na.rm = TRUE)
```

(Alternatively, a tapply()/sapply() matrix is acceptable—no packages.)

Constraints

- **Base R only**; **no external packages** or file I/O.
- Use **data frames** as the primary structure (no tibbles).
- Prefer **vectorized** computations; avoid loops.
- Keep code **commented** and variable names **clear**.

Program

```
# -------------------------------------
# Task 1 — Create Column Vectors
# -------------------------------------

PatientID <- c("HOS1001","HOS1002","HOS1003","HOS1004","HOS1005",
               "HOS1006","HOS1007","HOS1008","HOS1009","HOS1010")

Name <- c("Arjun Menon","Bhavana Iyer","Chirag Gupta","Devika Nair","Eshwar Rao",
          "Farida Sheikh","Gaurav Kumar","Harini Krishnan","Ishita S","Jatin Verma")

Department <- c("Cardiology","Neurology","Endocrinology","Pulmonology","Cardiology",

"Endocrinology","Neurology","Pulmonology","Cardiology","Endocrinology")

Diagnosis <- c("Hypertension","Migraine","Type-2 Diabetes","Asthma","Hypertension",
               "Thyroid Disorder","Epilepsy","COPD","Arrhythmia","Type-1 Diabetes")

Test1 <- c(82,71,90,64,95,60,78,68,87,83)
Test2 <- c(76,69,92,70,94,63,82,72,85,79)
Test3 <- c(88,73,86,67,93,61,79,70,90,77)
Test4 <- c(84,75,91,72,96,66,81,69,88,85)
```

```
# Duplicate column
Test3_Score <- Test3


# -------------------------------------
# Task 2 — Build the Data Frame
# -------------------------------------

patients <- data.frame(
  PatientID,
  Name,
  Department,
  Diagnosis,
  Test1,
  Test2,
  Test3,
  Test4,
  Test3_Score,
  stringsAsFactors = FALSE
)

# Display structure and first 6 rows
str(patients)
head(patients)


# -------------------------------------
# Task 3 — Patient-Level Sum, Average, Rank
# -------------------------------------

# Sum of test scores
patients$Sum <- rowSums(patients[,c("Test1","Test2","Test3","Test4")])

# Average score
patients$Average <- round(patients$Sum/4,2)

# Rank (highest sum gets rank 1)
patients$Rank <- rank(-patients$Sum, ties.method = "min")

# Print full dataframe
print(patients)

# Sorted by rank
patients_sorted <- patients[order(patients$Rank),]
print(patients_sorted)

# Note: ties.method="min" means equal sums receive the same lowest rank.
```

```
# -------------------------------------
# Task 4 — Test-Wise Average (Overall)
# -------------------------------------

test_cols <- c("Test1","Test2","Test3","Test4")

test_wise_avg <- colMeans(patients[,test_cols], na.rm = TRUE)

print(test_wise_avg)


# -------------------------------------
# Task 5 — Department-Wise Averages
# -------------------------------------

dept_test_avg <- aggregate(
  patients[,test_cols],
  by = list(Department = patients$Department),
  FUN = function(x) mean(x, na.rm = TRUE)
)

# Overall average across tests
dept_test_avg$Overall_Average <- rowMeans(dept_test_avg[,test_cols], na.rm =
TRUE)

# Sort by overall average (highest first)
dept_test_avg_sorted <- dept_test_avg[order(-dept_test_avg$Overall_Average),]

print(dept_test_avg_sorted)
```

Output Snapshots

```
  Test3_Score Sum Average Rank
           93 378   94.50    1
           86 359   89.75    2
           90 350   87.50    3
           88 330   82.50    4
           77 324   81.00    5
           79 320   80.00    6
           73 288   72.00    7
           70 279   69.75    8
           67 273   68.25    9
           61 250   62.50   10
```

```
> print(test_wise_avg)
Test1 Test2 Test3 Test4
 77.8  78.2  78.4  80.7
>
```

```
> print(dept_test_avg_sorted)
     Department    Test1 Test2    Test3    Test4 Overall_Average
1    Cardiology 88.00000  85.0 90.33333 89.33333        88.16667
2 Endocrinology 77.66667  78.0 74.66667 80.66667        77.75000
3     Neurology 74.50000  75.5 76.00000 78.00000        76.00000
4   Pulmonology 66.00000  71.0 68.50000 70.50000        69.00000
>
```

Explanation/ Observations

1. Creation of Vectors

- Separate vectors were created for each column using the `c()` function in R.
- Character vectors were used for:
  - PatientID
  - Name
  - Department
  - Diagnosis
- Numeric vectors were used for the diagnostic test scores:
  - Test1
  - Test2
  - Test3
  - Test4
- A duplicate vector **Test3_Score** was created from **Test3** as required in the question.

2. Creation of Data Frame

- All the vectors were combined into a **data frame named** `patients` using the `data.frame()` function.
- The argument `stringsAsFactors = FALSE` was used to keep text values as **character data instead of factors**.
- `str(patients)` was used to **check the structure of the data frame**.
- `head(patients)` was used to **display the first 6 rows of the dataset**.

3. Calculation of Sum, Average, and Rank

- A new column **Sum** was created to calculate the **total score of each patient across the four diagnostic tests**.
- The `rowSums()` function was used to compute this efficiently.
- Another column **Average** was created by **dividing the Sum by 4** to calculate the average test score.
- The **rank() function** was used to assign ranks based on the **Sum column**.
- The negative sign `-patients$Sum` ensures that **higher scores receive Rank 1**.
- `ties.method = "min"` ensures that **patients with equal scores receive the same minimum rank**.
- The data frame was also **sorted using** `order(patients$Rank)` to display patients from **highest rank to lowest rank**.

4. Test-Wise Average

- The overall average score for each diagnostic test (**Test1, Test2, Test3, Test4**) was calculated across all patients.

- The `colMeans()` function was used to compute the **average of each test column**.
- This provides a quick overview of **overall performance for each diagnostic test**.

---

5. Department-Wise Averages

- The **aggregate() function** was used to calculate the **average test scores for each department**.
- The mean score for **Test1, Test2, Test3, and Test4** was calculated for every department.
- An additional column **Overall_Average** was created using `rowMeans()` to calculate the **average of all four tests for each department**.
- The results were optionally **sorted by Overall_Average in descending order** to identify which department has the highest overall test performance.

References